

Lab Exercise 14- Vulnerability Testing of a Spring Boot Application Using SonarQube

Objective

In this lab, you will learn how to:

- Create a Spring Boot application with intentional vulnerabilities
 - Run static code analysis using SonarQube
 - Identify security vulnerabilities and code smells
 - Review SonarQube findings
 - Understand secure coding practices
-

Prerequisites

- Java 17 or later
 - Maven
 - IntelliJ IDEA
 - Docker
 - SonarQube running locally
-

Step 1: Start SonarQube

Run SonarQube using Docker:

```
docker run -d --name sonarqube -p 9000:9000 sonarqube:lts-community
```

Verify:

```
docker ps
```

Access SonarQube:

http://localhost:9000

Default credentials:

Username: admin

Password: admin

Step 2: Create a Spring Boot Project

Open IntelliJ IDEA.

Create a new Spring Boot project:

Project Name: **VulnerabilityDemo**

Group: **com.example**

Artifact: **vulnerabilitydemo**

Java Version: **17**

Dependencies:

Spring Web

Step 3: Configure SonarQube

Add the following plugin to pom.xml:

```
<dependencies>
<dependency>
  <groupId>junit</groupId>
  <artifactId>junit</artifactId>
  <version>4.12</version>
  <scope>test</scope>
```

```
</dependency>
<dependency>
  <groupId>org.sonarsource.scanner.maven</groupId>
  <artifactId>sonar-maven-plugin</artifactId>
  <version>3.9.0.2155</version>
</dependency>
</dependencies>

<profiles>
  <profile>
    <id>sonar</id>
    <activation>
      <activeByDefault>true</activeByDefault>
    </activation>
    <properties>
      <sonar.host.url>http://13.127.10.201:9000</sonar.host.url>
    </properties>
  </profile>
</profiles>
```

Step 4: Create Vulnerable Class

Create the file:

VulnerableApplication.java

```
package com.example.vulnerabilitydemo;
import java.sql.Connection;
import java.sql.ResultSet;
import java.sql.Statement;
import java.util.Random;

public class VulnerableApplication {

    private static final String USERNAME = "admin";
    private static final String PASSWORD = "Admin123";

    private Connection connection;

    public VulnerableApplication(Connection connection) {
        this.connection = connection;
    }

    public void findUser(String username) throws Exception {

        String query =
            "SELECT * FROM users WHERE username="
            + username + "'";

        Statement stmt = connection.createStatement();

        ResultSet rs = stmt.executeQuery(query);

        while (rs.next()) {
            System.out.println(rs.getString("username"));
        }
    }
}
```

```
public int generateToken() {  
  
    Random random = new Random();  
  
    return random.nextInt();  
}  
  
public void divideNumbers() {  
  
    try {  
        int result = 100 / o;  
    } catch (Exception e) {  
  
    }  
}  
  
public void printCredentials() {  
  
    System.out.println(USERNAME);  
    System.out.println(PASSWORD);  
}  
}
```

Step 5: Build the Project

Open IntelliJ Terminal:

```
mvn clean install
```

Expected Output:

BUILD SUCCESS

Step 6: Run SonarQube Analysis

```
mvn install sonar:sonar
```

Step 7: Review Findings

Open:

```
http://localhost:9000
```

Select project:

```
vulnerabilitydemo
```

Navigate to:

```
Issues
```

```
Security Hotspots
```

```
Code Smells
```

1. SQL Injection Vulnerability

```
import java.sql.Connection;
import java.sql.ResultSet;
import java.sql.Statement;

public class UserService {

    private Connection connection;

    public UserService(Connection connection) {
        this.connection = connection;
    }
}
```

```
public void findUser(String username) throws Exception {

    String query =
        "SELECT * FROM users WHERE username='"
        + username + "'";

    Statement stmt = connection.createStatement();

    ResultSet rs = stmt.executeQuery(query);

    while (rs.next()) {
        System.out.println(rs.getString("username"));
    }
}
```

SonarQube Findings

- SQL Injection
 - Security Hotspot
 - User-controlled input used in SQL query
-

2. Hardcoded Credentials

```
public class DatabaseConfig {

    private static final String DB_URL =
        "jdbc:mysql://localhost:3306/company";

    private static final String USERNAME =
        "admin";

    private static final String PASSWORD =
        "Admin123";

    public void connect() {
```

```
        System.out.println("Connecting to DB...");
    }
}
```

SonarQube Findings

- Hardcoded credentials
 - Security Hotspot
 - Sensitive information in source code
-

3. Weak Random Number Generator

```
import java.util.Random;

public class TokenGenerator {

    public int generateToken() {

        Random random = new Random();

        return random.nextInt();
    }
}
```

SonarQube Findings

- Predictable random number generation
 - Suggestion to use SecureRandom
-

4. Empty Catch Block

```
public class ExceptionDemo {  
  
    public void divide() {  
  
        try {  
            int x = 10 / 0;  
        }  
        catch (Exception e) {  
        }  
    }  
}
```

SonarQube Findings

- Empty catch block
 - Exception ignored
-

5. Hardcoded API Key

```
public class ApiClient {  
  
    private static final String API_KEY =  
        "1234567890-SECRET-KEY";  
  
    public void callApi() {  
  
        System.out.println(API_KEY);  
    }  
}
```

SonarQube Findings

- Hardcoded secret
- Security Hotspot